Problem Statement:

Expense Report Agent

Problem Statement

Design an AI agent for the following scenario:

"Generate my travel expense report for last month and email it to finance."

Your agent should:

• Retrieve expenses

• Filter date

• Categorize expenses

• Calculate totals

• Generate PDF

• Email report

• Handle missing data

CLASS=AIA_7

GROUP NO-4

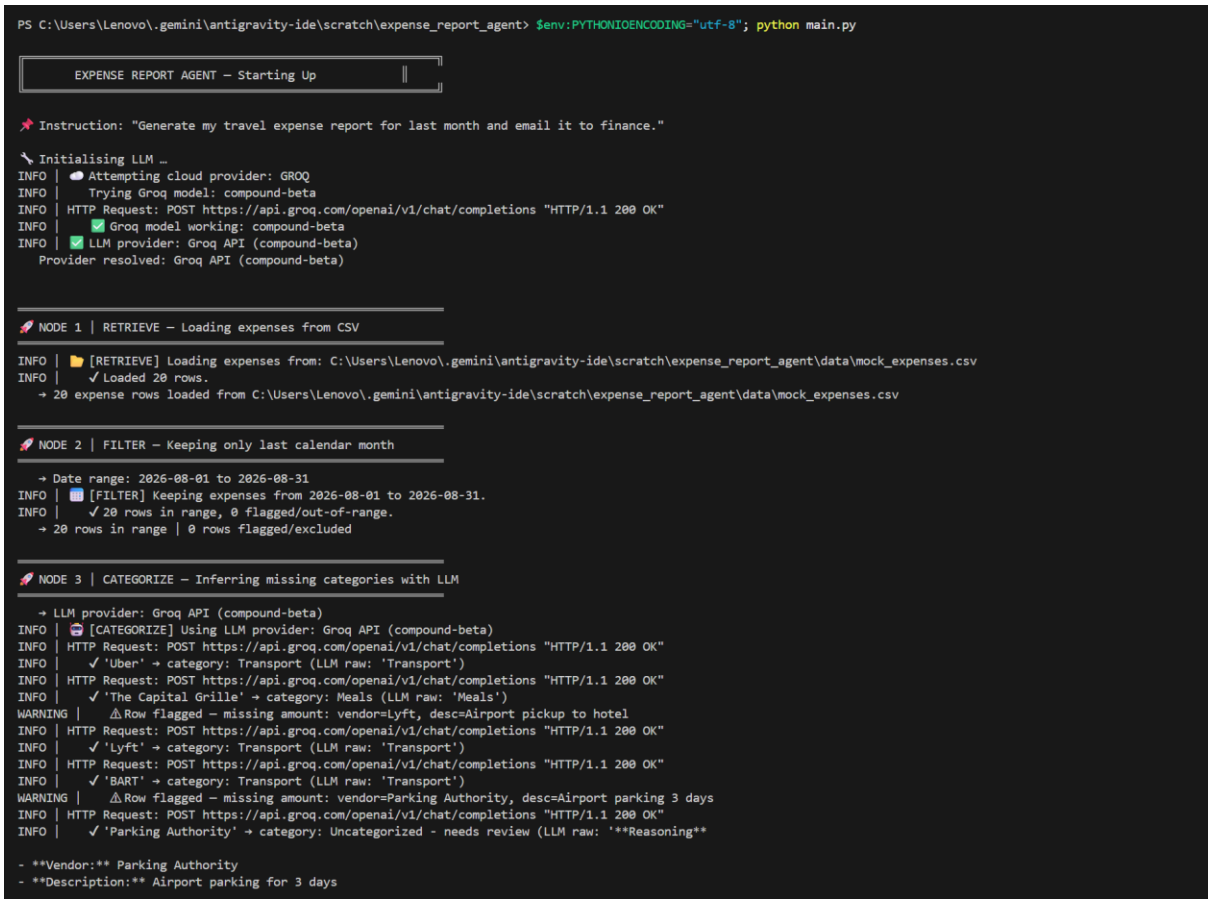
NAMES-Aryan Maradwar_11

Asta Joseph_12

Dhiraj Sarawale_20

Ridhima Maliya_49

Output:



```
PS C:\Users\Lenovo\.gemini\antigravity-ide\scratch\expense_report_agent> $env:PYTHONIOENCODING="utf-8"; python main.py

╔══════════════════════════════════════════════════╗
║       EXPENSE REPORT AGENT — Starting Up         ║
╚══════════════════════════════════════════════════╝

📌 Instruction: "Generate my travel expense report for last month and email it to finance."

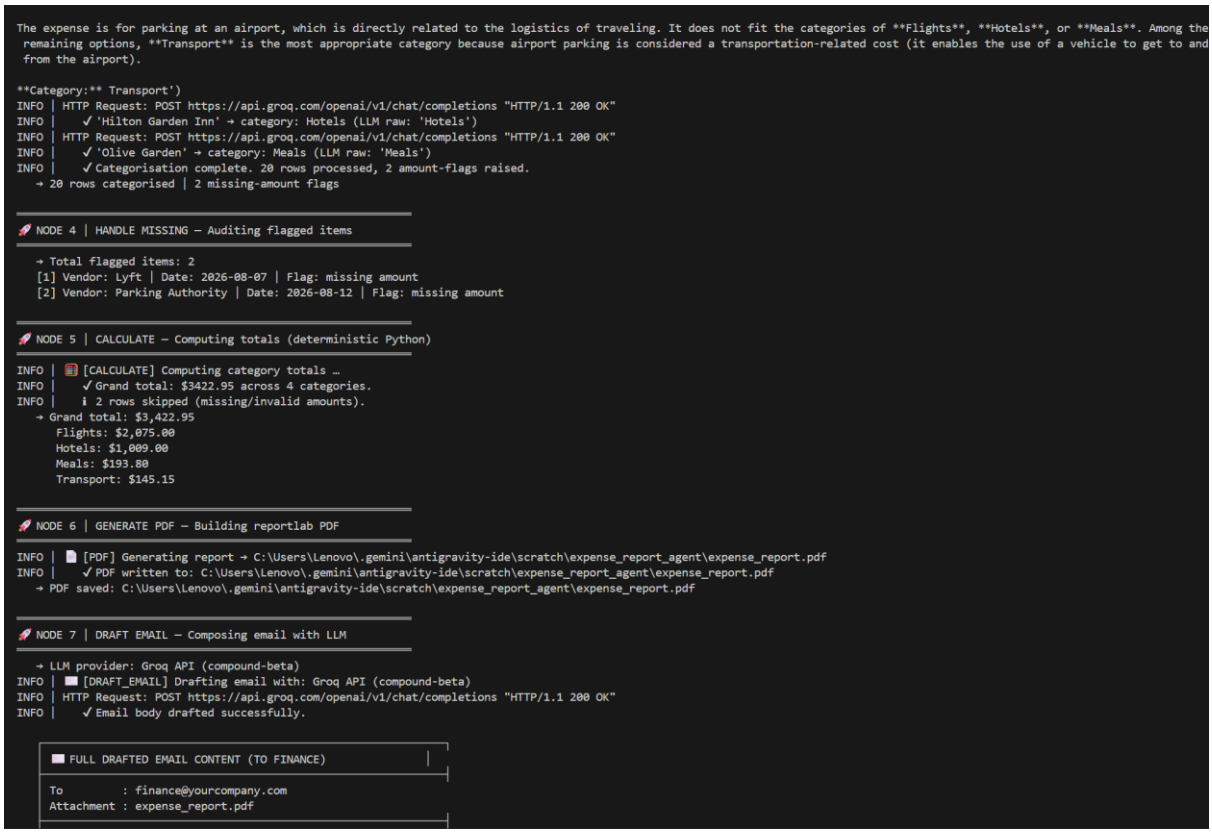
🔧 Initialising LLM …
INFO | 🚀 Attempting cloud provider: GROQ
INFO |    Trying Groq model: compound-beta
INFO | HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
INFO |    ✅ Groq model working: compound-beta
INFO | ✅ LLM provider: Groq API (compound-beta)
   Provider resolved: Groq API (compound-beta)


──────────────────────────────────────────────────
🚀 NODE 1 | RETRIEVE — Loading expenses from CSV
──────────────────────────────────────────────────
INFO | 📂 [RETRIEVE] Loading expenses from: C:\Users\Lenovo\.gemini\antigravity-ide\scratch\expense_report_agent\data\mock_expenses.csv
INFO |    ✓ Loaded 20 rows.
   → 20 expense rows loaded from C:\Users\Lenovo\.gemini\antigravity-ide\scratch\expense_report_agent\data\mock_expenses.csv

──────────────────────────────────────────────────
🚀 NODE 2 | FILTER — Keeping only last calendar month
──────────────────────────────────────────────────
   → Date range: 2026-08-01 to 2026-08-31
INFO | 🗓️ [FILTER] Keeping expenses from 2026-08-01 to 2026-08-31.
INFO |    ✓ 20 rows in range, 0 flagged/out-of-range.
   → 20 rows in range | 0 rows flagged/excluded

──────────────────────────────────────────────────
🚀 NODE 3 | CATEGORIZE — Inferring missing categories with LLM
──────────────────────────────────────────────────
   → LLM provider: Groq API (compound-beta)
INFO | 🤖 [CATEGORIZE] Using LLM provider: Groq API (compound-beta)
INFO | HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
INFO |    ✓ 'Uber' → category: Transport (LLM raw: 'Transport')
INFO | HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
INFO |    ✓ 'The Capital Grille' → category: Meals (LLM raw: 'Meals')
WARNING |    ⚠️ Row flagged — missing amount: vendor=Lyft, desc=Airport pickup to hotel
INFO | HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
INFO |    ✓ 'Lyft' → category: Transport (LLM raw: 'Transport')
INFO | HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
INFO |    ✓ 'BART' → category: Transport (LLM raw: 'Transport')
WARNING |    ⚠️ Row flagged — missing amount: vendor=Parking Authority, desc=Airport parking 3 days
INFO | HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
INFO |    ✓ 'Parking Authority' → category: Uncategorized - needs review (LLM raw: '**Reasoning**

- **Vendor:** Parking Authority
- **Description:** Airport parking for 3 days
```



```
The expense is for parking at an airport, which is directly related to the logistics of traveling. It does not fit the categories of **Flights**, **Hotels**, or **Meals**. Among the
 remaining options, **Transport** is the most appropriate category because airport parking is considered a transportation-related cost (it enables the use of a vehicle to get to and
 from the airport).

**Category:** Transport')
INFO | HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
INFO |    ✓ 'Hilton Garden Inn' → category: Hotels (LLM raw: 'Hotels')
INFO | HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
INFO |    ✓ 'Olive Garden' → category: Meals (LLM raw: 'Meals')
INFO |    ✓ Categorisation complete. 20 rows processed, 2 amount-flags raised.
   → 20 rows categorised | 2 missing-amount flags

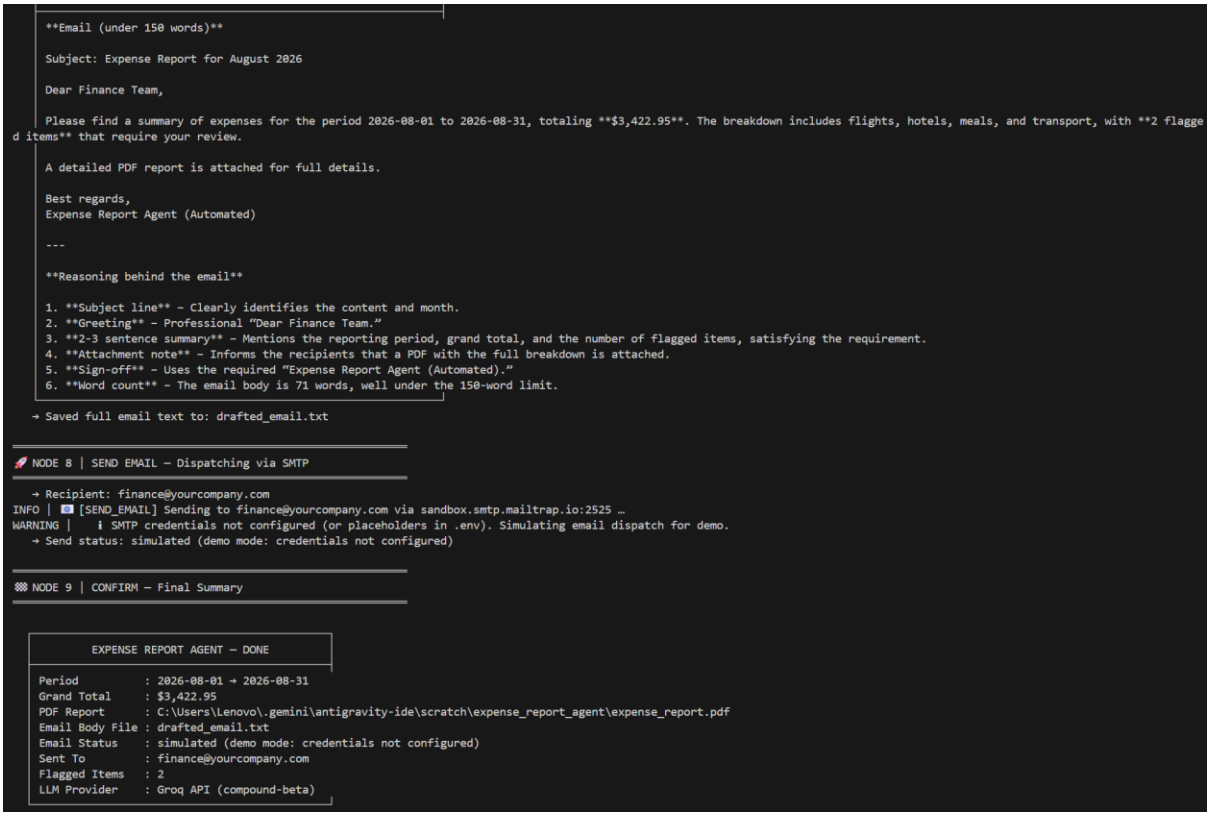
──────────────────────────────────────────────────
🚀 NODE 4 | HANDLE MISSING — Auditing flagged items
──────────────────────────────────────────────────
   → Total flagged items: 2
   [1] Vendor: Lyft | Date: 2026-08-07 | Flag: missing amount
   [2] Vendor: Parking Authority | Date: 2026-08-12 | Flag: missing amount

──────────────────────────────────────────────────
🚀 NODE 5 | CALCULATE — Computing totals (deterministic Python)
──────────────────────────────────────────────────
INFO | 🧮 [CALCULATE] Computing category totals …
INFO |    ✓ Grand total: $3422.95 across 4 categories.
INFO |    ℹ 2 rows skipped (missing/invalid amounts).
   → Grand total: $3,422.95
      Flights: $2,075.00
      Hotels: $1,009.00
      Meals: $193.80
      Transport: $145.15

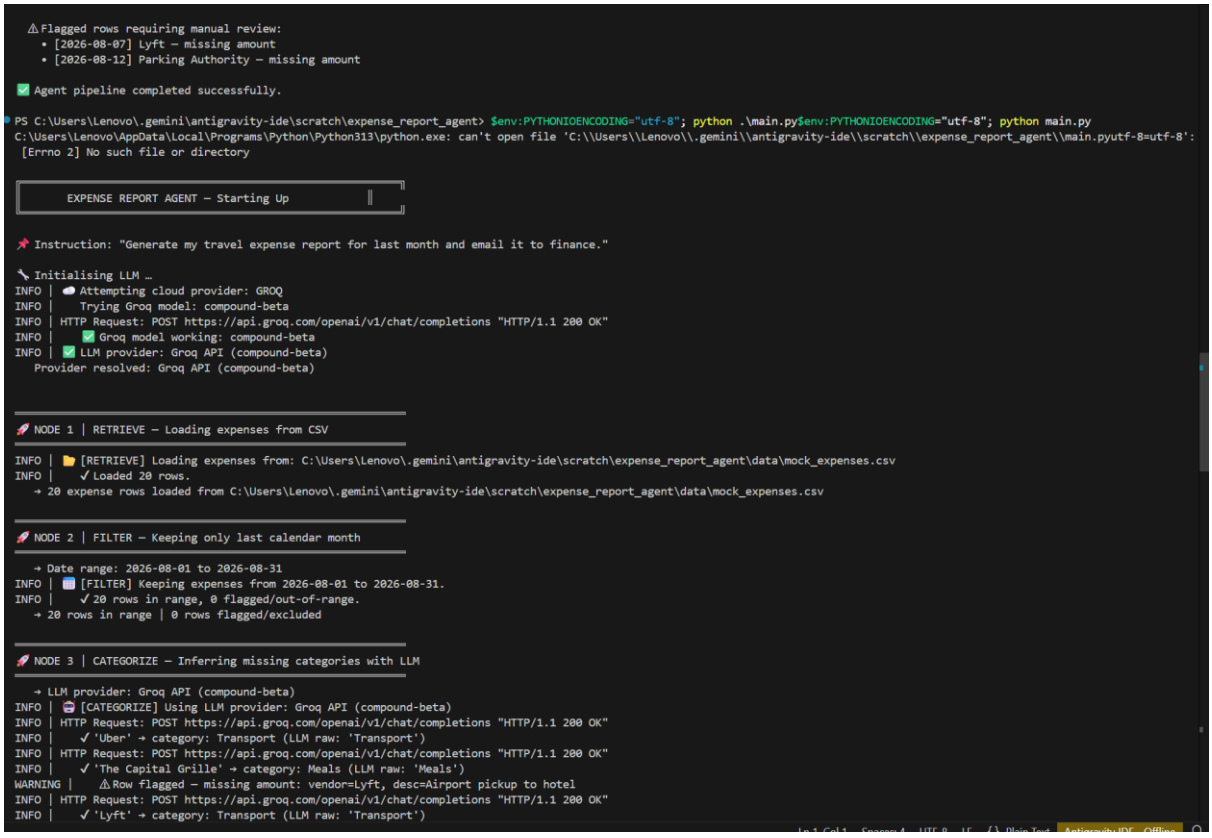
──────────────────────────────────────────────────
🚀 NODE 6 | GENERATE PDF — Building reportlab PDF
──────────────────────────────────────────────────
INFO | 📄 [PDF] Generating report → C:\Users\Lenovo\.gemini\antigravity-ide\scratch\expense_report_agent\expense_report.pdf
INFO |    ✓ PDF written to: C:\Users\Lenovo\.gemini\antigravity-ide\scratch\expense_report_agent\expense_report.pdf
   → PDF saved: C:\Users\Lenovo\.gemini\antigravity-ide\scratch\expense_report_agent\expense_report.pdf

──────────────────────────────────────────────────
🚀 NODE 7 | DRAFT EMAIL — Composing email with LLM
──────────────────────────────────────────────────
   → LLM provider: Groq API (compound-beta)
INFO | ✉️ [DRAFT_EMAIL] Drafting email with: Groq API (compound-beta)
INFO | HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
INFO |    ✓ Email body drafted successfully.

┌──────────────────────────────────────────────────┐
│ ✉️ FULL DRAFTED EMAIL CONTENT (TO FINANCE)       │
├──────────────────────────────────────────────────┤
│ To         : finance@yourcompany.com
│ Attachment : expense_report.pdf
└──────────────────────────────────────────────────┘
```


Email (under 150 words)
Subject: Expense Report for August 2026
Dear Finance Team,
Please find a summary of expenses for the period 2026-08-01 to 2026-08-31, totaling **$3,422.95**. The breakdown includes flights, hotels, meals, and transport, with **2 flagged items** that require your review.
A detailed PDF report is attached for full details.
Best regards,
Expense Report Agent (Automated)

Reasoning behind the email
1. **Subject line** – Clearly identifies the content and month.
2. **Greeting** – Professional "Dear Finance Team."
3. **2-3 sentence summary** – Mentions the reporting period, grand total, and the number of flagged items, satisfying the requirement.
4. **Attachment note** – Informs the recipients that a PDF with the full breakdown is attached.
5. **Sign-off** – Uses the required "Expense Report Agent (Automated)."
6. **Word count** – The email body is 71 words, well under the 150-word limit.
→ Saved full email text to: drafted_email.txt
NODE 8 | SEND EMAIL — Dispatching via SMTP
→ Recipient: finance@yourcompany.com
INFO | [SEND_EMAIL] Sending to finance@yourcompany.com via sandbox.smtp.mailtrap.io:2525 …
WARNING | ℹ SMTP credentials not configured (or placeholders in .env). Simulating email dispatch for demo.
→ Send status: simulated (demo mode: credentials not configured)
NODE 9 | CONFIRM — Final Summary
EXPENSE REPORT AGENT — DONE
Period : 2026-08-01 → 2026-08-31
Grand Total : $3,422.95
PDF Report : C:\Users\Lenovo\.gemini\antigravity-ide\scratch\expense_report_agent\expense_report.pdf
Email Body File : drafted_email.txt
Email Status : simulated (demo mode: credentials not configured)
Sent To : finance@yourcompany.com
Flagged Items : 2
LLM Provider : Groq API (compound-beta)


⚠ Flagged rows requiring manual review:
• [2026-08-07] Lyft — missing amount
• [2026-08-12] Parking Authority — missing amount
✅ Agent pipeline completed successfully.
PS C:\Users\Lenovo\.gemini\antigravity-ide\scratch\expense_report_agent> $env:PYTHONIOENCODING="utf-8"; python .\main.py$env:PYTHONIOENCODING="utf-8"; python main.py
C:\Users\Lenovo\AppData\Local\Programs\Python\Python313\python.exe: can't open file 'C:\\Users\\Lenovo\\.gemini\\antigravity-ide\\scratch\\expense_report_agent\\main.pyutf-8=utf-8': [Errno 2] No such file or directory
EXPENSE REPORT AGENT — Starting Up
📌 Instruction: "Generate my travel expense report for last month and email it to finance."
🔧 Initialising LLM …
INFO | Attempting cloud provider: GROQ
INFO | Trying Groq model: compound-beta
INFO | HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
INFO | ✅ Groq model working: compound-beta
INFO | ✅ LLM provider: Groq API (compound-beta)
Provider resolved: Groq API (compound-beta)
NODE 1 | RETRIEVE — Loading expenses from CSV
INFO | [RETRIEVE] Loading expenses from: C:\Users\Lenovo\.gemini\antigravity-ide\scratch\expense_report_agent\data\mock_expenses.csv
INFO | ✓ Loaded 20 rows.
→ 20 expense rows loaded from C:\Users\Lenovo\.gemini\antigravity-ide\scratch\expense_report_agent\data\mock_expenses.csv
NODE 2 | FILTER — Keeping only last calendar month
→ Date range: 2026-08-01 to 2026-08-31
INFO | [FILTER] Keeping expenses from 2026-08-01 to 2026-08-31.
INFO | ✓ 20 rows in range, 0 flagged/out-of-range.
→ 20 rows in range | 0 rows flagged/excluded
NODE 3 | CATEGORIZE — Inferring missing categories with LLM
→ LLM provider: Groq API (compound-beta)
INFO | [CATEGORIZE] Using LLM provider: Groq API (compound-beta)
INFO | HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
INFO | ✓ 'Uber' → category: Transport (LLM raw: 'Transport')
INFO | HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
INFO | ✓ 'The Capital Grille' → category: Meals (LLM raw: 'Meals')
WARNING | ⚠ Row flagged — missing amount: vendor=Lyft, desc=Airport pickup to hotel
INFO | HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
INFO | ✓ 'Lyft' → category: Transport (LLM raw: 'Transport')

```
PS C:\Users\Lenovo\.gemini\antigravity-ide\scratch\expense_report_agent> $env:PYTHONIOENCODING="utf-8"; python .\main.py$env:PYTHONIOENCODING="utf-8"; python main.py
INFO | HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
INFO |    ✓ 'The Capital Grille' → category: Meals (LLM raw: 'Meals')
WARNING |    ⚠ Row flagged — missing amount: vendor=Lyft, desc=Airport pickup to hotel
INFO | HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
INFO |    ✓ 'Lyft' → category: Transport (LLM raw: 'Transport')
INFO | HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
INFO |    ✓ 'BART' → category: Transport (LLM raw: 'Transport')
WARNING |    ⚠ Row flagged — missing amount: vendor=Parking Authority, desc=Airport parking 3 days
INFO | HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
INFO |    ✓ 'Parking Authority' → category: Transport (LLM raw: 'Transport')
INFO | HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
INFO |    ✓ 'Hilton Garden Inn' → category: Misc (LLM raw: 'Misc')
INFO | HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
INFO |    ✓ 'Olive Garden' → category: Uncategorized - needs review (LLM raw: '**Reasoning:**
- The vendor is Olive Garden, a well-known restaurant chain.
- The description "Team dinner - project debrief" indicates the expense was for a meal taken by the team.
- Among the provided categories (Flights, Hotels, Meals, Transport, Misc), the expense clearly aligns with food/drink costs incurred during business travel.

**Category:**
Meals')
INFO |    ✓ Categorisation complete. 20 rows processed, 2 amount-flags raised.
   → 20 rows categorised | 2 missing-amount flags

──────────────────────────────────────────────────
🚀 NODE 4 | HANDLE MISSING — Auditing flagged items
──────────────────────────────────────────────────
   → Total flagged items: 2
   [1] Vendor: Lyft | Date: 2026-08-07 | Flag: missing amount
   [2] Vendor: Parking Authority | Date: 2026-08-12 | Flag: missing amount

──────────────────────────────────────────────────
🚀 NODE 5 | CALCULATE — Computing totals (deterministic Python)
──────────────────────────────────────────────────
INFO | 🧮 [CALCULATE] Computing category totals …
INFO |    ✓ Grand total: $3422.95 across 6 categories.
INFO |    ℹ 2 rows skipped (missing/invalid amounts).
   → Grand total: $3,422.95
      Flights: $2,075.00
      Hotels: $974.00
      Meals: $138.80
      Misc: $35.00
      Transport: $145.15
      Uncategorized - needs review: $55.00

──────────────────────────────────────────────────
🚀 NODE 6 | GENERATE PDF — Building reportlab PDF
──────────────────────────────────────────────────
INFO | 📄 [PDF] Generating report → C:\Users\Lenovo\.gemini\antigravity-ide\scratch\expense_report_agent\expense_report.pdf
INFO |    ✓ PDF written to: C:\Users\Lenovo\.gemini\antigravity-ide\scratch\expense_report_agent\expense_report.pdf
   → PDF saved: C:\Users\Lenovo\.gemini\antigravity-ide\scratch\expense_report_agent\expense_report.pdf
```

```
      Uncategorized - needs review: $55.00

──────────────────────────────────────────────────
🚀 NODE 6 | GENERATE PDF — Building reportlab PDF
──────────────────────────────────────────────────
INFO | 📄 [PDF] Generating report → C:\Users\Lenovo\.gemini\antigravity-ide\scratch\expense_report_agent\expense_report.pdf
INFO |    ✓ PDF written to: C:\Users\Lenovo\.gemini\antigravity-ide\scratch\expense_report_agent\expense_report.pdf
   → PDF saved: C:\Users\Lenovo\.gemini\antigravity-ide\scratch\expense_report_agent\expense_report.pdf

──────────────────────────────────────────────────
🚀 NODE 7 | DRAFT EMAIL — Composing email with LLM
──────────────────────────────────────────────────
   → LLM provider: Groq API (compound-beta)
INFO | ✉️ [DRAFT_EMAIL] Drafting email with: Groq API (compound-beta)
INFO | HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
INFO |    ✓ Email body drafted successfully.

┌──────────────────────────────────────────────────┐
│ ✉️ FULL DRAFTED EMAIL CONTENT (TO FINANCE)       │
├──────────────────────────────────────────────────┤
│ To         : finance@yourcompany.com
│ Attachment : expense_report.pdf
├──────────────────────────────────────────────────┤
│ Subject: August 2026 Travel Expense Report

│ Dear Finance Team,

│ Please find attached the travel expense report for August 2026 (2026-08-01 to 2026-08-31). The total expenses amount to **$3,422.95**, and there are **2 flagged items** that ne
ed review. The PDF contains the full category breakdown and details.

│ Best regards,
│ Expense Report Agent (Automated)

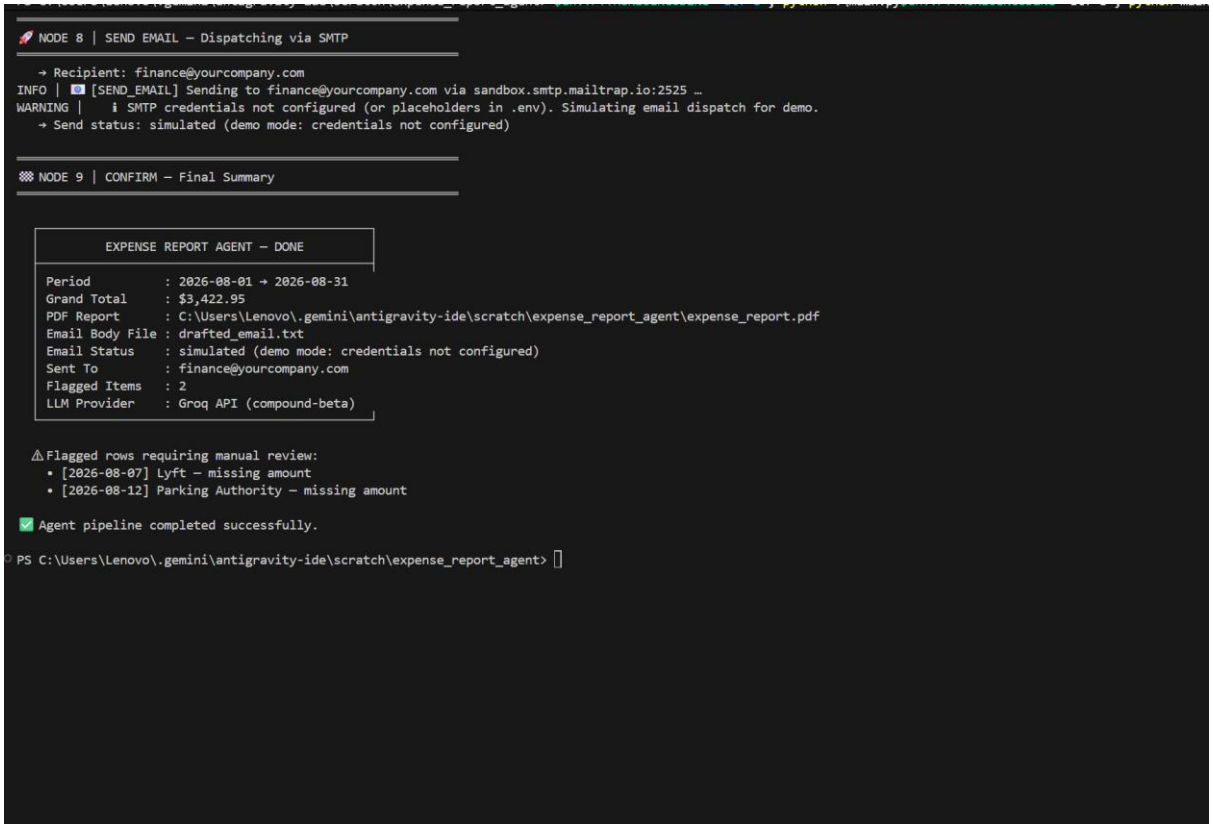
│ ---

│ **Reasoning and Information Used**

│ - **Period covered:** 2026-08-01 to 2026-08-31
│ - **Grand total:** $3,422.95
│ - **Category breakdown:** Flights $2,075.00; Hotels $974.00; Meals $138.80; Misc $35.00; Transport $145.15; Uncategorized $55.00 (needs review)
│ - **Flagged/exception items:** 2
│ - **Attachment:** PDF report with full details

│ The email follows the user's requirements: a clear subject line, professional greeting, a 2-sentence summary mentioning total, period, and flagged items, a note about the attac
hed PDF, and a professional sign-off, all under 150 words.
└──────────────────────────────────────────────────┘
   → Saved full email text to: drafted_email.txt

──────────────────────────────────────────────────
🚀 NODE 8 | SEND EMAIL — Dispatching via SMTP
──────────────────────────────────────────────────
   → Recipient: finance@yourcompany.com
```


🚀 NODE 8 | SEND EMAIL — Dispatching via SMTP
→ Recipient: finance@yourcompany.com
INFO | 📧 [SEND_EMAIL] Sending to finance@yourcompany.com via sandbox.smtp.mailtrap.io:2525 …
WARNING | ℹ SMTP credentials not configured (or placeholders in .env). Simulating email dispatch for demo.
→ Send status: simulated (demo mode: credentials not configured)
🏁 NODE 9 | CONFIRM — Final Summary
EXPENSE REPORT AGENT — DONE
Period : 2026-08-01 → 2026-08-31
Grand Total : $3,422.95
PDF Report : C:\Users\Lenovo\.gemini\antigravity-ide\scratch\expense_report_agent\expense_report.pdf
Email Body File : drafted_email.txt
Email Status : simulated (demo mode: credentials not configured)
Sent To : finance@yourcompany.com
Flagged Items : 2
LLM Provider : Groq API (compound-beta)
⚠ Flagged rows requiring manual review:
• [2026-08-07] Lyft — missing amount
• [2026-08-12] Parking Authority — missing amount
✅ Agent pipeline completed successfully.
PS C:\Users\Lenovo\.gemini\antigravity-ide\scratch\expense_report_agent>

```
"""
llm_config.py
_____________
Single source-of-truth for LLM initialisation.

Priority chain (controlled by env vars):
  LLM_MODE=api_first   → try cloud provider → fall back to local Ollama
  LLM_MODE=local_only  → skip cloud entirely → use local Ollama

Cloud provider (when LLM_MODE=api_first):
  LLM_PROVIDER=groq       → Groq (tries multiple model names for compatibility)
  LLM_PROVIDER=anthropic  → Anthropic Claude claude-3-haiku-20240307
  LLM_PROVIDER=openai     → OpenAI GPT-4o-mini
"""

import os
import logging

from dotenv import load_dotenv

load_dotenv()

logger = logging.getLogger(__name__)
logging.basicConfig(level=logging.INFO, format="%(levelname)s | %(message)s")

# Groq model names to try in order (fallback list — Groq deprecates models regularly)
_GROQ_MODELS = [
    "compound-beta",                    # confirmed working (2026)
    "compound-beta-mini",               # lighter version
    "deepseek-r1-distill-llama-70b",    # DeepSeek R1
    "qwen-qwq-32b",                     # Qwen reasoning model
```

```
    "llama-3.1-8b-instant",          # original default (may be deprecated)
    "llama3-8b-8192",                # legacy name
    "gemma2-9b-it",                  # Google Gemma
]

# ──────────────────────────────────────────────────────────────────────────────
# Internal helpers
# ──────────────────────────────────────────────────────────────────────────────

def _try_groq():
    """Attempt to create a Groq ChatLLM. Tries multiple model names. Returns (llm, label) or raises."""
    api_key = os.getenv("GROQ_API_KEY", "").strip()
    if not api_key:
        raise EnvironmentError("GROQ_API_KEY not set or empty.")

    from langchain_groq import ChatGroq  # type: ignore

    last_error = None
    for model_name in _GROQ_MODELS:
        try:
            logger.info("   Trying Groq model: %s", model_name)
            llm = ChatGroq(
                model=model_name,
                groq_api_key=api_key,
                temperature=0.2,
            )
            # Lightweight connectivity + model availability check
            llm.invoke("hi")
            label = f"Groq API ({model_name})"
            logger.info("   ✅ Groq model working: %s", model_name)
            return llm, label
```

```
            except Exception as e:
                logger.warning("   ⚠ Groq model '%s' unavailable: %s", model_name, str(e)[:120])
                last_error = e

        raise RuntimeError(f"All Groq models failed. Last error: {last_error}")

    def _try_anthropic():
        """Attempt to create an Anthropic ChatLLM. Returns (llm, label) or raises."""
        api_key = os.getenv("ANTHROPIC_API_KEY", "").strip()
        if not api_key:
            raise EnvironmentError("ANTHROPIC_API_KEY not set or empty.")

        from langchain_anthropic import ChatAnthropic  # type: ignore
        llm = ChatAnthropic(
            model="claude-3-haiku-20240307",
            anthropic_api_key=api_key,
            temperature=0.2,
        )
        llm.invoke("hi")
        return llm, "Anthropic API (claude-3-haiku-20240307)"

    def _try_openai():
        """Attempt to create an OpenAI ChatLLM. Returns (llm, label) or raises."""
        api_key = os.getenv("OPENAI_API_KEY", "").strip()
        if not api_key:
            raise EnvironmentError("OPENAI_API_KEY not set or empty.")

        from langchain_openai import ChatOpenAI  # type: ignore
        llm = ChatOpenAI(
            model="gpt-4o-mini",
            openai_api_key=api_key,
```

```
        temperature=0.2,
    )
    llm.invoke("hi")
    return llm, "OpenAI API (gpt-4o-mini)"

def _try_ollama():
    """
    Attempt to create a local Ollama ChatLLM.
    Tries langchain_ollama package first (newer), then langchain_community fallback.
    Returns (llm, label) or raises.
    """
    model = os.getenv("OLLAMA_MODEL", "llama3.1:8b")
    base_url = os.getenv("OLLAMA_BASE_URL", "http://localhost:11434")

    # Try the dedicated langchain_ollama package first (preferred in newer versions)
    try:
        from langchain_ollama import ChatOllama  # type: ignore
        logger.info("   Using langchain_ollama.ChatOllama")
    except ImportError:
        # Fall back to langchain_community
        try:
            from langchain_community.chat_models import ChatOllama  # type: ignore
            logger.info("   Using langchain_community ChatOllama (legacy)")
        except ImportError:
            raise ImportError(
                "Cannot import ChatOllama. Install with: "
                "pip install langchain-ollama   OR   pip install langchain-community"
            )

    llm = ChatOllama(
        model=model,
```

```
        base_url=base_url,
        temperature=0.2,
    )
    # Quick connectivity check
    llm.invoke("hi")
    return llm, f"Local Ollama ({model})"

# ——————————————————————————————————————————————————————————————————————

# Public entry point

# ——————————————————————————————————————————————————————————————————————

def get_llm():
    """
    Returns a ready-to-use LangChain chat model.

    Resolution order:
      1. If LLM_MODE=local_only  → Ollama only
      2. If LLM_MODE=api_first (default):
          a. Try the cloud provider specified by LLM_PROVIDER
          b. On any failure, fall back to local Ollama

    Never raises — if all providers fail, returns (None, label) and logs the error.
    """
    llm_mode     = os.getenv("LLM_MODE", "api_first").strip().lower()
    llm_provider = os.getenv("LLM_PROVIDER", "groq").strip().lower()

    # ── Local-only mode ──────────────────────────────────────────────────
    if llm_mode == "local_only":
        logger.info(" 🔒 LLM_MODE=local_only — skipping all cloud APIs.")
        try:
            llm, label = _try_ollama()
```

```
        logger.info("✅ LLM provider: %s", label)
        return llm, label
    except Exception as exc:
        logger.error("❌ Local Ollama failed: %s", exc)
        logger.error(
            "   Is Ollama running? Try: ollama serve  &&  ollama pull llama3.1:8b"
        )
        return None, "None — all providers failed"

# ── API-first mode ──────────────────────────────────────────────────────
provider_map = {
    "groq":      _try_groq,
    "anthropic": _try_anthropic,
    "openai":    _try_openai,
}

cloud_fn = provider_map.get(llm_provider)
if cloud_fn is None:
    logger.warning(
        "⚠️  Unknown LLM_PROVIDER='%s'. Defaulting to Groq.", llm_provider
    )
    cloud_fn = _try_groq

logger.info("☁️  Attempting cloud provider: %s", llm_provider.upper())
try:
    llm, label = cloud_fn()
    logger.info("✅ LLM provider: %s", label)
    return llm, label
except Exception as exc:
    logger.warning(
        "⚠️  Cloud provider '%s' failed: %s — falling back to local Ollama.",
```

```
            llm_provider, exc,
        )

    # ── Ollama fallback ──────────────────────────────────────────────────────
    logger.info("🔄 Falling back to Local Ollama …")
    try:
        llm, label = _try_ollama()
        logger.info("✅ LLM provider (fallback): %s", label)
        return llm, label
    except Exception as exc:
        logger.error("❌ Local Ollama fallback also failed: %s", exc)
        logger.error(
            "   Make sure Ollama is running and the model is pulled:\n"
            "   ollama serve\n"
            "   ollama pull llama3.1:8b"
        )
        return None, "None — all providers failed"



"""
main.py

NOTE: Run with PYTHONIOENCODING=utf-8 on Windows to support emoji print output:
  $env:PYTHONIOENCODING="utf-8"; python main.py   (PowerShell)
  set PYTHONIOENCODING=utf-8 && python main.py    (CMD)
────────

LangGraph pipeline definition for the Expense Report Agent.

Graph nodes (in execution order):
  retrieve      → load CSV
```

```
  filter       → keep last calendar month
  categorize   → LLM: infer missing categories
  handle_missing→ consolidate all exception rows
  calculate    → deterministic totals
  generate_pdf → reportlab PDF
  draft_email  → LLM: compose email body
  send_email   → SMTP dispatch
  confirm      → print final summary

Run:
  python main.py
"""

from __future__ import annotations

import logging
import os
from calendar import monthrange
from datetime import date, timedelta
from pathlib import Path
from typing import Any, Optional

import sys

from dotenv import load_dotenv
from typing_extensions import TypedDict

# ── Windows UTF-8 fix (handles emoji/Unicode in print statements) ─────────────
if sys.stdout.encoding and sys.stdout.encoding.lower() not in ("utf-8", "utf8"):
    sys.stdout.reconfigure(encoding="utf-8", errors="replace")
if sys.stderr.encoding and sys.stderr.encoding.lower() not in ("utf-8", "utf8"):
```

```
    sys.stderr.reconfigure(encoding="utf-8", errors="replace")

from langgraph.graph import StateGraph, END

from llm_config import get_llm
from tools import (
    retrieve_expenses,
    filter_last_month,
    categorize_expenses,
    calculate_totals,
    generate_pdf_report,
    draft_email,
    send_email_with_pdf,
)

load_dotenv()

logging.basicConfig(
    level=logging.INFO,
    format="%(levelname)s | %(name)s | %(message)s",
)
logger = logging.getLogger("main")

# ═══════════════════════════════════════════════════════════════════════════
# State Schema
# ═══════════════════════════════════════════════════════════════════════════

class ExpenseState(TypedDict, total=False):
    """Shared state passed between every LangGraph node."""
```

```
    # Inputs
    instruction: str              # Original user instruction
    csv_path:    str              # Path to mock CSV

    # Data
    raw_expenses:        list[dict]   # All rows from CSV
    filtered_expenses:   list[dict]   # Rows in last calendar month
    categorized_expenses: list[dict]  # All rows with category filled in
    exceptions:          list[dict]   # Flagged rows (missing data)

    # Computed
    totals:      dict[str, Any]   # {category_totals, grand_total}
    date_range:  tuple[str, str]  # (first_day_str, last_day_str)

    # Outputs
    pdf_path:    str    # Absolute path to generated PDF
    email_body:  str    # Drafted email text
    email_result: dict  # Send result (status, recipient, …)

    # LLM info
    llm:         Any    # The chat model instance
    llm_label:   str    # Human-readable provider label

#
═══════════════════════════════════════════════════════════════════════════════
# Helper: compute last-month date range strings
#
═══════════════════════════════════════════════════════════════════════════════

def _last_month_range() -> tuple[str, str]:
```

```
    today = date.today()
    first_of_this = today.replace(day=1)
    last_day_prev = first_of_this - timedelta(days=1)
    first_day_prev = last_day_prev.replace(day=1)
    return first_day_prev.strftime("%Y-%m-%d"), last_day_prev.strftime("%Y-%m-%d")

#
═══════════════════════════════════════════════════════════════════════════════
# Node 1 — Retrieve
#
═══════════════════════════════════════════════════════════════════════════════

def node_retrieve(state: ExpenseState) -> ExpenseState:
    print("\n" + "═" * 60)
    print(" NODE 1 | RETRIEVE — Loading expenses from CSV")
    print("═" * 60)

    csv_path = state.get("csv_path", "data/mock_expenses.csv")
    raw = retrieve_expenses(csv_path)

    print(f"   → {len(raw)} expense rows loaded from {csv_path}")
    return {**state, "raw_expenses": raw}

#
═══════════════════════════════════════════════════════════════════════════════
# Node 2 — Filter
#
═══════════════════════════════════════════════════════════════════════════════

def node_filter(state: ExpenseState) -> ExpenseState:
```

```
    print("\n" + "═" * 60)
    print(" NODE 2 | FILTER — Keeping only last calendar month")
    print("═" * 60)

    date_range = _last_month_range()
    print(f"   → Date range: {date_range[0]} to {date_range[1]}")

    valid, date_exceptions = filter_last_month(state["raw_expenses"])

    print(f"   → {len(valid)} rows in range | {len(date_exceptions)} rows flagged/excluded")
    return {
        **state,
        "filtered_expenses": valid,
        "exceptions":       date_exceptions,
        "date_range":       date_range,
    }

# ═══════════════════════════════════════════════════════════════════════════
# Node 3 — Categorise (LLM)
# ═══════════════════════════════════════════════════════════════════════════

def node_categorize(state: ExpenseState) -> ExpenseState:
    print("\n" + "═" * 60)
    print(" NODE 3 | CATEGORIZE — Inferring missing categories with LLM")
    print("═" * 60)

    llm      = state.get("llm")
    llm_label = state.get("llm_label", "Unknown")
    print(f"   → LLM provider: {llm_label}")
```

```
    categorized, amount_exceptions = categorize_expenses(
        state["filtered_expenses"], llm, llm_label
    )

    existing_exceptions = state.get("exceptions", [])
    all_exceptions = existing_exceptions + amount_exceptions

    print(f"   → {len(categorized)} rows categorised | {len(amount_exceptions)} missing-amount flags")
    return {
        **state,
        "categorized_expenses": categorized,
        "exceptions":           all_exceptions,
    }

# ═══════════════════════════════════════════════════════════════════════════
# Node 4 — Handle Missing Data (consolidation / audit log)
# ═══════════════════════════════════════════════════════════════════════════

def node_handle_missing(state: ExpenseState) -> ExpenseState:
    print("\n" + "═" * 60)
    print(" NODE 4 | HANDLE MISSING — Auditing flagged items")
    print("═" * 60)

    exceptions = state.get("exceptions", [])
    print(f"   → Total flagged items: {len(exceptions)}")
    for i, exc in enumerate(exceptions, 1):
        print(
            f"   [{i}] Vendor: {exc.get('vendor','?')} | "
```

```
            f"Date: {exc.get('date','?')} | "
            f"Flag: {exc.get('_flag','?')}"
        )

    if not exceptions:
        print("  ✓ No exceptions — all rows processed cleanly.")

    return state  # no state change, just audit

#
═══════════════════════════════════════════════════════════════════════════
═══════════════════════════
# Node 5 — Calculate (deterministic, NO LLM)
#
═══════════════════════════════════════════════════════════════════════════
═══════════════════════════

def node_calculate(state: ExpenseState) -> ExpenseState:
    print("\n" + "═" * 60)
    print(" NODE 5 | CALCULATE — Computing totals (deterministic Python)")
    print("═" * 60)

    totals = calculate_totals(state["categorized_expenses"])

    print(f"  → Grand total: ${totals['grand_total']:,.2f}")
    for cat, amt in sorted(totals["category_totals"].items()):
        print(f"     {cat}: ${amt:,.2f}")

    return {**state, "totals": totals}

#
═══════════════════════════════════════════════════════════════════════════
═══════════════════════════
```

```
# Node 6 — Generate PDF
#
═══════════════════════════════════════════════════════════════════════════════

def node_generate_pdf(state: ExpenseState) -> ExpenseState:
    print("\n" + "═" * 60)
    print("NODE 6 | GENERATE PDF — Building reportlab PDF")
    print("═" * 60)

    output_path = str(
        Path(__file__).parent / "expense_report.pdf"
    )

    pdf_path = generate_pdf_report(
        rows       = state["categorized_expenses"],
        totals     = state["totals"],
        exceptions = state.get("exceptions", []),
        date_range = state["date_range"],
        output_path= output_path,
    )

    print(f"   → PDF saved: {pdf_path}")
    return {**state, "pdf_path": pdf_path}

#
═══════════════════════════════════════════════════════════════════════════════
# Node 7 — Draft Email (LLM)
#
═══════════════════════════════════════════════════════════════════════════════
```

```
def node_draft_email(state: ExpenseState) -> ExpenseState:
    print("\n" + "═" * 60)
    print(" NODE 7 | DRAFT EMAIL — Composing email with LLM")
    print("═" * 60)

    llm      = state.get("llm")
    llm_label = state.get("llm_label", "Unknown")
    print(f"   → LLM provider: {llm_label}")

    body = draft_email(
        totals        = state["totals"],
        date_range    = state["date_range"],
        num_exceptions = len(state.get("exceptions", [])),
        llm           = llm,
        llm_label     = llm_label,
        instruction   = state.get("instruction", "Generate my travel expense report for last month and email it to finance."),
    )

    # Save drafted email to file for easy inspection/reading
    email_txt_path = Path(__file__).parent / "drafted_email.txt"
    recipient = os.getenv("FINANCE_EMAIL", "finance@yourcompany.com")
    with open(email_txt_path, "w", encoding="utf-8") as f:
        f.write(f"To: {recipient}\n")
        f.write(f"Attachment: {state.get('pdf_path', 'expense_report.pdf')}\n")
        f.write("–" * 60 + "\n")
        f.write(body + "\n")

    # Print the entire drafted email clearly in terminal
    print("\n  ┌──────────────────────────────────────────────────────────────┐")
    print("  │  FULL DRAFTED EMAIL CONTENT (TO FINANCE)                     │")
```

```
    print("    ├──────────────────────────────────────────────────────────────┤ ")
    print(f"    | To       : {recipient}")
    print(f"    | Attachment : {Path(state.get('pdf_path', 'expense_report.pdf')).name}")
    print("    ├──────────────────────────────────────────────────────────────┤ ")
    for line in body.split("\n"):
        print(f"    | {line}")
    print("    └──────────────────────────────────────────────────────────────┘ ")
    print(f"    → Saved full email text to: {email_txt_path.name}")
    return {**state, "email_body": body}


#
═══════════════════════════════════════════════════════════════════════════
# Node 8 — Send Email
#
═══════════════════════════════════════════════════════════════════════════


def node_send_email(state: ExpenseState) -> ExpenseState:
    print("\n" + "═" * 60)
    print(" NODE 8 | SEND EMAIL — Dispatching via SMTP")
    print("═" * 60)

    recipient = os.getenv("FINANCE_EMAIL", "finance@yourcompany.com")
    print(f"    → Recipient: {recipient}")

    result = send_email_with_pdf(
        email_body = state["email_body"],
        pdf_path   = state["pdf_path"],
        recipient  = recipient,
    )
```

```
    print(f"   → Send status: {result['status']}")
    return {**state, "email_result": result}

#
═══════════════════════════════════════════════════════════════════════════════
═══════════════════════════
# Node 9 — Confirm (final summary)
#
═══════════════════════════════════════════════════════════════════════════════
═══════════════════════════

def node_confirm(state: ExpenseState) -> ExpenseState:
    print("\n" + "═" * 60)
    print(" NODE 9 | CONFIRM — Final Summary")
    print("═" * 60)

    totals     = state.get("totals", {})
    exceptions = state.get("exceptions", [])
    result     = state.get("email_result", {})
    date_range = state.get("date_range", ("?", "?"))

    print(f"""
  ┌──────────────────────────────────────────────┐
  │        EXPENSE REPORT AGENT — DONE           │
  ├──────────────────────────────────────────────┤
  │ Period         : {date_range[0]} → {date_range[1]}
  │ Grand Total    : ${totals.get('grand_total', 0):,.2f}
  │ PDF Report     : {state.get('pdf_path', 'N/A')}
  │ Email Body File : drafted_email.txt
  │ Email Status   : {result.get('status', 'N/A')}
  │ Sent To        : {result.get('recipient', 'N/A')}
  │ Flagged Items  : {len(exceptions)}
```

```
  | LLM Provider    : {state.get('llm_label', 'N/A')}
  └──────────────────────────────────────────────────┘""")

    if exceptions:
        print("\n  ⚠ Flagged rows requiring manual review:")
        for exc in exceptions:
            print(
                f"    • [{exc.get('date','?')}] {exc.get('vendor','?')} — "
                f"{exc.get('_flag','?')}"
            )

    print("\n Agent pipeline completed successfully.\n")
    return state


# ═══════════════════════════════════════════════════════════════════════════
# Build the LangGraph
# ═══════════════════════════════════════════════════════════════════════════

def build_graph():
    graph = StateGraph(ExpenseState)

    # Register nodes
    graph.add_node("retrieve",       node_retrieve)
    graph.add_node("filter",         node_filter)
    graph.add_node("categorize",     node_categorize)
    graph.add_node("handle_missing", node_handle_missing)
    graph.add_node("calculate",      node_calculate)
    graph.add_node("generate_pdf",   node_generate_pdf)
```

```
    graph.add_node("draft_email",    node_draft_email)
    graph.add_node("send_email",     node_send_email)
    graph.add_node("confirm",        node_confirm)

    # Define edges (sequential pipeline)
    graph.set_entry_point("retrieve")
    graph.add_edge("retrieve",       "filter")
    graph.add_edge("filter",         "categorize")
    graph.add_edge("categorize",     "handle_missing")
    graph.add_edge("handle_missing", "calculate")
    graph.add_edge("calculate",      "generate_pdf")
    graph.add_edge("generate_pdf",   "draft_email")
    graph.add_edge("draft_email",    "send_email")
    graph.add_edge("send_email",     "confirm")
    graph.add_edge("confirm",        END)

    return graph.compile()

# ═══════════════════════════════════════════════════════════════════════════
# Entry point
# ═══════════════════════════════════════════════════════════════════════════

def main():
    print("\n" + "╔" + "═" * 58 + "╗")
    print("║       EXPENSE REPORT AGENT — Starting Up            ║")
    print("╚" + "═" * 58 + "╝")

    # Accept custom instruction via command-line arg, or use default
    if len(sys.argv) > 1:
```

```
    instruction = " ".join(sys.argv[1:]).strip()
  else:
    instruction = (
      "Generate my travel expense report for last month "
      "and email it to finance."
    )
  print(f"\n Instruction: \"{instruction}\"\n")

  # — Initialise LLM ————————————————————————————————————————
  print(" 🔧 Initialising LLM …")
  llm, llm_label = get_llm()
  print(f"   Provider resolved: {llm_label}\n")

  # — Build & run graph ————————————————————————————————————————
  app = build_graph()

  csv_path = str(Path(__file__).parent / "data" / "mock_expenses.csv")

  initial_state: ExpenseState = {
    "instruction": instruction,
    "csv_path":    csv_path,
    "llm":         llm,
    "llm_label":   llm_label,
  }

  final_state = app.invoke(initial_state)
  return final_state

if __name__ == "__main__":
  main()
```

"""

tools.py

_____

All pure-Python tool functions used by the LangGraph nodes.

Deterministic tools (no LLM):

  retrieve_expenses   – load CSV from disk

  filter_last_month   – keep only last calendar month

  calculate_totals   – sum amounts per category + grand total

LLM-powered tools:

  categorize_expenses – infer category from vendor/description

  draft_email   – compose professional email body

Side-effect tools:

  generate_pdf_report – render reportlab PDF

  send_email_with_pdf – SMTP dispatch with attachment

"""

```
from __future__ import annotations

import csv
import io
import logging
import sys
import os
import smtplib
from calendar import monthrange
from datetime import date, datetime
from email import encoders
from email.mime.base import MIMEBase
```

```
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from pathlib import Path
from typing import Any

from dotenv import load_dotenv

load_dotenv()

# ── Windows UTF-8 fix ──────────────────────────────────────────────
if sys.stdout.encoding and sys.stdout.encoding.lower() not in ("utf-8", "utf8"):
    sys.stdout.reconfigure(encoding="utf-8", errors="replace")
if sys.stderr.encoding and sys.stderr.encoding.lower() not in ("utf-8", "utf8"):
    sys.stderr.reconfigure(encoding="utf-8", errors="replace")

logger = logging.getLogger(__name__)

# ═══════════════════════════════════════════════════════════════════
# TOOL 1 — Retrieve expenses from CSV
# ═══════════════════════════════════════════════════════════════════

def retrieve_expenses(csv_path: str) -> list[dict[str, Any]]:
    """Load expenses from the mock CSV file. Returns raw rows as dicts."""
    logger.info("📂 [RETRIEVE] Loading expenses from: %s", csv_path)
    path = Path(csv_path)
    if not path.exists():
        raise FileNotFoundError(f"CSV file not found: {csv_path}")
```

```
    rows: list[dict[str, Any]] = []
    with open(path, newline="", encoding="utf-8") as fh:
        reader = csv.DictReader(fh)
        for row in reader:
            # Normalise whitespace
            clean = {k.strip(): v.strip() for k, v in row.items()}
            rows.append(clean)

    logger.info("  ✓ Loaded %d rows.", len(rows))
    return rows

#
═══════════════════════════════════════════════════════════════════════════
═══════════════════════════

# TOOL 2 — Filter to last calendar month (deterministic Python)
#
═══════════════════════════════════════════════════════════════════════════
═══════════════════════════


def filter_last_month(rows: list[dict]) -> tuple[list[dict], list[dict]]:
    """
    Keep only rows whose date falls in the previous calendar month.

    Returns:
        (valid_rows, exception_rows)
        exception_rows contain rows with missing/unparseable dates.
    """
    today = date.today()
    # Compute first and last day of last month
    first_of_this_month = today.replace(day=1)
    last_month_last_day = first_of_this_month - __import__("datetime").timedelta(days=1)
    last_month_first_day = last_month_last_day.replace(day=1)
```

```
logger.info(
    "🗓️ [FILTER] Keeping expenses from %s to %s.",
    last_month_first_day,
    last_month_last_day,
)

valid: list[dict] = []
exceptions: list[dict] = []

for row in rows:
    raw_date = row.get("date", "").strip()
    if not raw_date:
        row["_flag"] = "missing date"
        exceptions.append(row)
        logger.warning("   ⚠ Row flagged — missing date: %s", row)
        continue

    try:
        expense_date = datetime.strptime(raw_date, "%Y-%m-%d").date()
    except ValueError:
        row["_flag"] = f"unparseable date: '{raw_date}'"
        exceptions.append(row)
        logger.warning("   ⚠ Row flagged — bad date format: %s", row)
        continue

    if last_month_first_day <= expense_date <= last_month_last_day:
        valid.append(row)

logger.info(
    "   ✓ %d rows in range, %d flagged/out-of-range.",
```

```
        len(valid), len(exceptions),
    )
    return valid, exceptions


#
═══════════════════════════════════════════════════════════════════════════
═══════════════════════════
# TOOL 3 — Categorise with LLM
#
═══════════════════════════════════════════════════════════════════════════
═══════════════════════════


VALID_CATEGORIES = {"Flights", "Hotels", "Meals", "Transport", "Misc"}


def categorize_expenses(
    rows: list[dict],
    llm,
    llm_label: str,
) -> tuple[list[dict], list[dict]]:
    """
    For rows with a missing or empty category, ask the LLM to infer it.
    Rows already categorised are passed through untouched.

    Returns:
        (categorized_rows, new_exceptions)
        new_exceptions: rows with a missing amount that get flagged here.
    """
    logger.info(" [CATEGORIZE] Using LLM provider: %s", llm_label)

    categorized: list[dict] = []
    new_exceptions: list[dict] = []

    for row in rows:
```

```
# — Flag missing amount but keep processing ——————————————————
if not row.get("amount", "").strip():
    row["_flag"] = "missing amount"
    new_exceptions.append({**row, "_source": "categorize_step"})
    logger.warning(
        "  ⚠ Row flagged — missing amount: vendor=%s, desc=%s",
        row.get("vendor"), row.get("description"),
    )
    # Still attempt to categorise for completeness

existing_cat = row.get("category", "").strip()
if existing_cat in VALID_CATEGORIES:
    # Already categorised — nothing to do
    categorized.append(row)
    continue

# — Ask LLM ——————————————————————————————————————————————————
vendor = row.get("vendor", "Unknown")
description = row.get("description", "")

prompt = (
    f"You are an expense categorisation assistant.\n"
    f"Classify the following business travel expense into EXACTLY ONE of these categories:\n"
    f"  Flights, Hotels, Meals, Transport, Misc\n\n"
    f"Expense:\n"
    f"  Vendor: {vendor}\n"
    f"  Description: {description}\n\n"
    f"Rules:\n"
    f"  - Reply with the category name ONLY (one word or two words exactly as listed).\n"
    f"  - If you cannot confidently infer the category, reply: Uncategorized - needs review\n"
)
```

```
if llm is None:
    inferred = "Uncategorized - needs review"
    logger.warning(
        "  ⚠ LLM unavailable — marking '%s' as Uncategorized.", vendor
    )
else:
    try:
        response = llm.invoke(prompt)
        raw_text = (
            response.content
            if hasattr(response, "content")
            else str(response)
        ).strip()

        # Validate — if LLM returned something unexpected, normalise
        if raw_text in VALID_CATEGORIES:
            inferred = raw_text
        elif "uncategorized" in raw_text.lower():
            inferred = "Uncategorized - needs review"
        else:
            # Try case-insensitive match
            match = next(
                (c for c in VALID_CATEGORIES if c.lower() == raw_text.lower()),
                None,
            )
            inferred = match if match else "Uncategorized - needs review"

        logger.info(
            "  ✓ '%s' → category: %s (LLM raw: '%s')",
            vendor, inferred, raw_text,
```

```
            )
        except Exception as exc:
            inferred = "Uncategorized - needs review"
            logger.warning("  ⚠ LLM call failed for '%s': %s", vendor, exc)

    row = {**row, "category": inferred}
    categorized.append(row)

logger.info(
    "  ✓ Categorisation complete. %d rows processed, %d amount-flags raised.",
    len(categorized), len(new_exceptions),
)
return categorized, new_exceptions


# ═══════════════════════════════════════════════════════════════════════════
# TOOL 4 — Calculate totals (deterministic Python, NO LLM)
# ═══════════════════════════════════════════════════════════════════════════

def calculate_totals(rows: list[dict]) -> dict[str, Any]:
    """
    Compute total per category and grand total.
    Rows with missing/invalid amounts are skipped (already flagged upstream).
    """
    logger.info(" [CALCULATE] Computing category totals …")

    category_totals: dict[str, float] = {}
    skipped = 0
```

```
    for row in rows:
        raw_amount = row.get("amount", "").strip()
        if not raw_amount:
            skipped += 1
            continue
        try:
            amount = float(raw_amount)
        except ValueError:
            logger.warning(
                "  Cannot parse amount '%s' for vendor '%s' — skipping.",
                raw_amount, row.get("vendor"),
            )
            skipped += 1
            continue

        cat = row.get("category", "Uncategorized - needs review")
        category_totals[cat] = category_totals.get(cat, 0.0) + amount

    grand_total = sum(category_totals.values())

    logger.info("  Grand total: $%.2f across %d categories.", grand_total, len(category_totals))
    if skipped:
        logger.info("  ℹ %d rows skipped (missing/invalid amounts).", skipped)

    return {
        "category_totals": category_totals,
        "grand_total": grand_total,
    }

#
═══════════════════════════════════════════════════════════════════════════
═══════════════════════════
```

```
# TOOL 5 — Generate PDF report with reportlab
#
═══════════════════════════════════════════════════════════════════════════════


def generate_pdf_report(
    rows: list[dict],
    totals: dict[str, Any],
    exceptions: list[dict],
    date_range: tuple[str, str],
    output_path: str,
) -> str:
    """
    Render a professional PDF expense report.

    Sections:
      • Header / metadata
      • Summary table (category → total)
      • Expense detail table
      • Exceptions appendix
    """
    logger.info("📄 [PDF] Generating report → %s", output_path)

    try:
        from reportlab.lib import colors
        from reportlab.lib.pagesizes import A4
        from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
        from reportlab.lib.units import cm
        from reportlab.platypus import (
            SimpleDocTemplate, Table, TableStyle, Paragraph,
            Spacer, HRFlowable, PageBreak,
        )
```

```
    from reportlab.lib.enums import TA_CENTER, TA_LEFT
except ImportError as exc:
    raise ImportError("reportlab is required. Run: pip install reportlab") from exc

doc = SimpleDocTemplate(
    output_path,
    pagesize=A4,
    rightMargin=2 * cm,
    leftMargin=2 * cm,
    topMargin=2.5 * cm,
    bottomMargin=2 * cm,
)

styles = getSampleStyleSheet()
story = []

# ── Colour palette ──────────────────────────────────────────────────────
BRAND_DARK   = colors.HexColor("#1a2744")
BRAND_MID    = colors.HexColor("#2e4a8e")
BRAND_LIGHT  = colors.HexColor("#dde8ff")
ACCENT_RED   = colors.HexColor("#c0392b")
ROW_ALT      = colors.HexColor("#f0f4ff")
WHITE        = colors.white
GREY_TEXT    = colors.HexColor("#555555")

# ── Custom paragraph styles ─────────────────────────────────────────────
title_style = ParagraphStyle(
    "Title",
    parent=styles["Heading1"],
    fontSize=22,
    textColor=BRAND_DARK,
```

```
    spaceAfter=4,
    alignment=TA_CENTER,
    fontName="Helvetica-Bold",
)
subtitle_style = ParagraphStyle(
    "Subtitle",
    parent=styles["Normal"],
    fontSize=11,
    textColor=GREY_TEXT,
    alignment=TA_CENTER,
    spaceAfter=18,
)
section_style = ParagraphStyle(
    "Section",
    parent=styles["Heading2"],
    fontSize=13,
    textColor=BRAND_MID,
    spaceBefore=16,
    spaceAfter=6,
    fontName="Helvetica-Bold",
)
normal_style = styles["Normal"]

# ── Header ──────────────────────────────────────────────────────────
story.append(Paragraph("Travel Expense Report", title_style))
story.append(Paragraph(
    f"Period: {date_range[0]} to {date_range[1]}"
    f"  |  Generated: {date.today().strftime('%B %d, %Y')}",
    subtitle_style,
))
story.append(HRFlowable(width="100%", thickness=2, color=BRAND_MID, spaceAfter=12))
```

```
# ── Summary table ──────────────────────────────────────────────────────
story.append(Paragraph("Summary by Category", section_style))

summary_data = [["Category", "Total Amount"]]
cat_totals = totals.get("category_totals", {})
for cat in sorted(cat_totals):
    summary_data.append([cat, f"${cat_totals[cat]:,.2f}"])
summary_data.append(["GRAND TOTAL", f"${totals.get('grand_total', 0):,.2f}"])

summary_table = Table(summary_data, colWidths=[11 * cm, 5 * cm])
summary_table.setStyle(TableStyle([
    # Header row
    ("BACKGROUND",   (0, 0), (-1, 0), BRAND_DARK),
    ("TEXTCOLOR",    (0, 0), (-1, 0), WHITE),
    ("FONTNAME",     (0, 0), (-1, 0), "Helvetica-Bold"),
    ("FONTSIZE",     (0, 0), (-1, 0), 11),
    ("ALIGN",        (1, 0), (1, -1), "RIGHT"),
    ("ALIGN",        (0, 0), (0, -1), "LEFT"),
    # Alternating rows
    *[
        ("BACKGROUND", (0, i), (-1, i), ROW_ALT if i % 2 == 0 else WHITE)
        for i in range(1, len(summary_data) - 1)
    ],
    # Grand total row
    ("BACKGROUND",   (0, -1), (-1, -1), BRAND_MID),
    ("TEXTCOLOR",    (0, -1), (-1, -1), WHITE),
    ("FONTNAME",     (0, -1), (-1, -1), "Helvetica-Bold"),
    ("FONTSIZE",     (0, -1), (-1, -1), 11),
    # Borders
    ("GRID",         (0, 0), (-1, -1), 0.5, colors.HexColor("#c0c8e0")),
```

```
    ("ROWBACKGROUND", (0, 0), (-1, 0), BRAND_DARK),
    ("BOTTOMPADDING", (0, 0), (-1, -1), 8),
    ("TOPPADDING",    (0, 0), (-1, -1), 8),
    ("LEFTPADDING",   (0, 0), (-1, -1), 10),
]))
story.append(summary_table)
story.append(Spacer(1, 12))

# ── Detail table ──────────────────────────────────────────────────────
story.append(Paragraph("Expense Detail", section_style))

detail_headers = ["Date", "Vendor", "Description", "Category", "Amount", "Receipt"]
detail_data = [detail_headers]
for row in rows:
    amount = row.get("amount", "").strip()
    amount_str = f"${float(amount):,.2f}" if amount else "MISSING"
    receipt = "✓" if row.get("receipt_attached", "").lower() == "true" else "✗"
    detail_data.append([
        row.get("date", ""),
        row.get("vendor", ""),
        (row.get("description", "")[:40] + "…")
        if len(row.get("description", "")) > 40 else row.get("description", ""),
        row.get("category", ""),
        amount_str,
        receipt,
    ])

col_widths = [2.2*cm, 3.5*cm, 5.5*cm, 3.0*cm, 2.0*cm, 1.5*cm]
detail_table = Table(detail_data, colWidths=col_widths, repeatRows=1)
detail_table.setStyle(TableStyle([
    ("BACKGROUND",    (0, 0), (-1, 0), BRAND_DARK),
```

```
    ("TEXTCOLOR",     (0, 0), (-1, 0), WHITE),
    ("FONTNAME",      (0, 0), (-1, 0), "Helvetica-Bold"),
    ("FONTSIZE",      (0, 0), (-1, -1), 8),
    ("ALIGN",         (4, 1), (4, -1), "RIGHT"),
    ("ALIGN",         (5, 0), (5, -1), "CENTER"),
    ("GRID",          (0, 0), (-1, -1), 0.4, colors.HexColor("#c0c8e0")),
    ("BOTTOMPADDING", (0, 0), (-1, -1), 6),
    ("TOPPADDING",    (0, 0), (-1, -1), 6),
    ("LEFTPADDING",   (0, 0), (-1, -1), 6),
    *[
        ("BACKGROUND", (0, i), (-1, i), ROW_ALT if i % 2 == 0 else WHITE)
        for i in range(1, len(detail_data))
    ],
]))
story.append(detail_table)

# ── Exceptions appendix ──────────────────────────────────────────────
if exceptions:
    story.append(PageBreak())
    story.append(Paragraph("Appendix: Flagged Items / Exceptions", section_style))
    story.append(Paragraph(
        "The following expense rows could not be fully processed and require manual review.",
        normal_style,
    ))
    story.append(Spacer(1, 8))

    exc_headers = ["Date", "Vendor", "Description", "Flag / Reason"]
    exc_data = [exc_headers]
    for exc_row in exceptions:
        exc_data.append([
            exc_row.get("date", "N/A"),
```

```
            exc_row.get("vendor", "N/A"),
            (exc_row.get("description", "")[:45] + "…")
            if len(exc_row.get("description", "")) > 45 else exc_row.get("description", ""),
            exc_row.get("_flag", "Unknown issue"),
        ])

    exc_col_widths = [2.2*cm, 3.5*cm, 6.0*cm, 5.5*cm]
    exc_table = Table(exc_data, colWidths=exc_col_widths, repeatRows=1)
    exc_table.setStyle(TableStyle([
        ("BACKGROUND",    (0, 0), (-1, 0), ACCENT_RED),
        ("TEXTCOLOR",     (0, 0), (-1, 0), WHITE),
        ("FONTNAME",      (0, 0), (-1, 0), "Helvetica-Bold"),
        ("FONTSIZE",      (0, 0), (-1, -1), 8),
        ("GRID",          (0, 0), (-1, -1), 0.4, colors.HexColor("#f0c0c0")),
        ("BACKGROUND",    (0, 1), (-1, -1), colors.HexColor("#fff5f5")),
        ("BOTTOMPADDING", (0, 0), (-1, -1), 6),
        ("TOPPADDING",    (0, 0), (-1, -1), 6),
        ("LEFTPADDING",   (0, 0), (-1, -1), 6),
    ]))
    story.append(exc_table)

# ── Footer note ──────────────────────────────────────────────────────
story.append(Spacer(1, 20))
story.append(HRFlowable(width="100%", thickness=1, color=BRAND_LIGHT))
story.append(Paragraph(
    "This report was generated automatically by the Expense Report Agent. "
    "Please contact finance@yourcompany.com for any discrepancies.",
    ParagraphStyle("Footer", parent=normal_style, fontSize=8, textColor=GREY_TEXT),
))

doc.build(story)
```

```
    logger.info("  ✓ PDF written to: %s", output_path)
    return output_path


# ═══════════════════════════════════════════════════════════════════════════
# TOOL 6 — Draft email (LLM)
# ═══════════════════════════════════════════════════════════════════════════

def draft_email(
    totals: dict[str, Any],
    date_range: tuple[str, str],
    num_exceptions: int,
    llm,
    llm_label: str,
    instruction: str = "Generate my travel expense report for last month and email it to finance.",
) -> str:
    """
    Use the LLM to compose a short professional email summarising the report.
    Falls back to a template if the LLM is unavailable.
    """
    logger.info("✉  [DRAFT_EMAIL] Drafting email with: %s", llm_label)

    grand_total = totals.get("grand_total", 0.0)
    cat_totals  = totals.get("category_totals", {})
    breakdown   = "\n".join(
        f"  • {cat}: ${amt:,.2f}" for cat, amt in sorted(cat_totals.items())
    )

    prompt = (
```

```
    f"You are a professional corporate assistant. Write a concise email to the finance team.\n\n"
    f"User Instruction:\n"
    f"  \"{instruction}\"\n\n"
    f"Context:\n"
    f"  - Period covered: {date_range[0]} to {date_range[1]}\n"
    f"  - Grand total of expenses: ${grand_total:,.2f}\n"
    f"  - Category breakdown:\n{breakdown}\n"
    f"  - Number of flagged/exception items requiring review: {num_exceptions}\n"
    f"  - A PDF report is attached.\n\n"
    f"Requirements for the email:\n"
    f"  - Follow any specific guidance requested in the User Instruction above\n"
    f"  - Subject line (write 'Subject: ...' on the first line)\n"
    f"  - Professional greeting\n"
    f"  - 2-3 sentence summary mentioning the total, period, and flagged items\n"
    f"  - A note that the PDF is attached for full details\n"
    f"  - Professional sign-off from 'Expense Report Agent (Automated)'\n"
    f"  - Keep it under 150 words total.\n"
)

fallback_body = (
    f"Subject: Travel Expense Report — {date_range[0]} to {date_range[1]}\n\n"
    f"Dear Finance Team,\n\n"
    f"Please find attached the travel expense report for the period "
    f"{date_range[0]} to {date_range[1]}.\n\n"
    f"The grand total for the period is ${grand_total:,.2f}. "
    f"There are {num_exceptions} item(s) flagged for manual review due to "
    f"missing data (amount or date). Please refer to the Exceptions appendix "
    f"in the attached PDF for details.\n\n"
    f"Kindly acknowledge receipt and process at your earliest convenience.\n\n"
    f"Best regards,\n"
    f"Expense Report Agent (Automated)"
```

```
    )

    if llm is None:
        logger.warning("  ⚠ LLM unavailable — using fallback email template.")
        return fallback_body

    import time
    for attempt in range(2):
        try:
            response = llm.invoke(prompt)
            body = (
                response.content
                if hasattr(response, "content")
                else str(response)
            ).strip()
            logger.info("  ✓ Email body drafted successfully.")
            return body
        except Exception as exc:
            if "429" in str(exc) and attempt == 0:
                logger.info("  ⏳ Rate limit pause (waiting 5s for token bucket)...")
                time.sleep(5)
                continue
            logger.warning("  ⚠ LLM email draft failed: %s — using fallback.", str(exc)[:120])
            return fallback_body

# ═══════════════════════════════════════════════════════════════════════════
# TOOL 7 — Send email via SMTP
# ═══════════════════════════════════════════════════════════════════════════
```

```
def send_email_with_pdf(
    email_body: str,
    pdf_path: str,
    recipient: str,
) -> dict[str, Any]:
    """
    Send the drafted email with the PDF attached via SMTP.
    Reads credentials from environment variables.
    Returns a result dict with status and details.
    """
    smtp_host = os.getenv("SMTP_HOST", "sandbox.smtp.mailtrap.io")
    smtp_port = int(os.getenv("SMTP_PORT", "2525"))
    smtp_user = os.getenv("SMTP_USER", "")
    smtp_pass = os.getenv("SMTP_PASS", "")
    from_addr = os.getenv("SMTP_FROM", "agent@expense-report.dev")

    logger.info(
        "📧 [SEND_EMAIL] Sending to %s via %s:%s …", recipient, smtp_host, smtp_port
    )

    # ── Parse subject from body (first line if it starts with "Subject:")
    lines = email_body.strip().split("\n")
    if lines and lines[0].lower().startswith("subject:"):
        subject = lines[0][len("subject:"):].strip()
        body_text = "\n".join(lines[1:]).strip()
    else:
        subject = "Travel Expense Report"
        body_text = email_body

    # ── Build MIME message ──────────────────────────────────────────
```

```
msg = MIMEMultipart()
msg["From"]    = from_addr
msg["To"]      = recipient
msg["Subject"] = subject
msg.attach(MIMEText(body_text, "plain"))

# ── Attach PDF ──────────────────────────────────────────────────────────
pdf_filename = Path(pdf_path).name
with open(pdf_path, "rb") as pdf_file:
    pdf_part = MIMEBase("application", "octet-stream")
    pdf_part.set_payload(pdf_file.read())
encoders.encode_base64(pdf_part)
pdf_part.add_header(
    "Content-Disposition",
    f'attachment; filename="{pdf_filename}"',
)
msg.attach(pdf_part)

# ── Send ────────────────────────────────────────────────────────────────
result: dict[str, Any] = {
    "status": "unknown",
    "recipient": recipient,
    "subject": subject,
    "smtp_host": smtp_host,
    "smtp_port": smtp_port,
    "pdf_attached": pdf_filename,
}

is_placeholder = (
    not smtp_user
    or not smtp_pass
```

```
    or "your_" in smtp_user.lower()
    or "your_" in smtp_pass.lower()
)
if is_placeholder:
    logger.warning(
        "  ℹ SMTP credentials not configured (or placeholders in .env). "
        "Simulating email dispatch for demo."
    )
    result["status"] = "simulated (demo mode: credentials not configured)"
    result["note"] = "Set real SMTP_USER and SMTP_PASS in .env to deliver real emails."
    return result

try:
    with smtplib.SMTP(smtp_host, smtp_port, timeout=15) as server:
        server.ehlo()
        server.starttls()
        server.login(smtp_user, smtp_pass)
        server.sendmail(from_addr, recipient, msg.as_string())
    result["status"] = "sent"
    logger.info("  ✓ Email sent successfully.")
except smtplib.SMTPAuthenticationError:
    logger.error("  ❌ SMTP authentication failed. Check SMTP_USER / SMTP_PASS.")
    result["status"] = "failed — SMTP auth error"
except Exception as exc:
    logger.error("  ❌ SMTP error: %s", exc)
    result["status"] = f"failed — {exc}"

return result
```